ADVANCED SQL

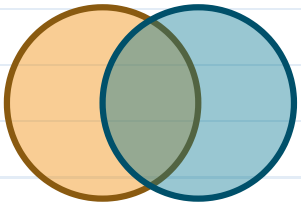
100 QUERY QUESTIONS

≋ (Written & Solved in MySQL 8) ≋

query.sql

```
SELECT dept, name,
RANK() OVER (
  PARTITION BY dept
  ORDER BY salary DESC)
FROM  employees;
```

dept	name	rnk
Sales	Priya	1



JOINS

WINDOWS

CTEs

GAPS & ISLANDS

FOUNDATIONS Q1–Q39

Joins & anti-joins · aggregation · conditional pivots · correlated subqueries · EXISTS vs IN

THE HEAVY STUFF Q40–Q79

CTEs & recursion · ranking · LAG/LEAD · frames & running totals · NTILE · top-N per group

THE CLOSERS Q80–Q100

Gaps & islands · streaks · de-duplication · NULL logic · date and time queries

VOLUME 1 OF 2 · Q1–Q100

Every question answered with a query you can actually run

👤➕ FOLLOW | 📤 SHARE | 💬 COMMENT | ❤️ LIKE

The Schema Behind All 100 Queries

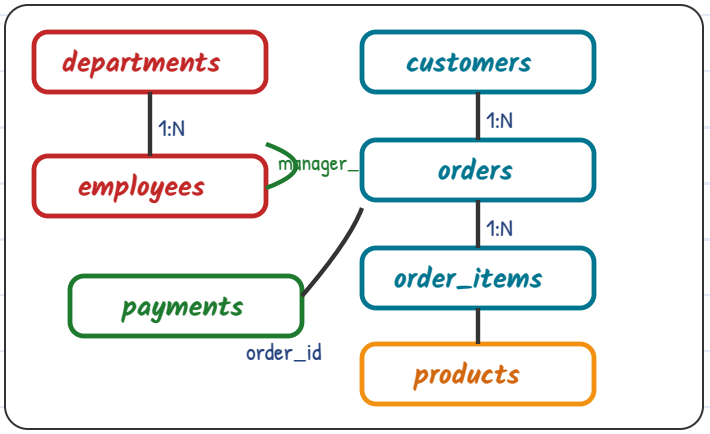
Every query in this deck runs on these 7 tables. Learn the schema once, then read the rest at speed. MySQL 8.0 syntax throughout.

Tables & Columns

```
employees(emp_id, emp_name, dept_id,
   manager_id, salary, hire_date, city)
departments(dept_id, dept_name, location)
customers(cust_id, cust_name, city,
   segment, signup_date)
orders(order_id, cust_id, order_date,
   status, amount)
order_items(item_id, order_id,
   product_id, qty, unit_price)
products(product_id, product_name,
   category, price)
payments(payment_id, order_id, paid_at,
   amount, method)
```

Note: manager_id points back at emp_id, which is what makes self-joins and recursion possible. status is one of PLACED, SHIPPED, DELIVERED, CANCELLED.

How The Tables Connect



Grain — Say It Before You Type

Table	One row per...
orders	order (header level, one customer)
order_items	product line inside an order
payments	payment attempt, so an order may have 0, 1 or many

Start Here — Q1 to Q3

Q1 **Count employees per department, keeping departments that have none.**

```
SELECT d.dept_name, COUNT(e.emp_id) AS emp_cnt
FROM departments d
LEFT JOIN employees e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name;
```

Trap: COUNT(*) returns 1 for an empty department because the LEFT JOIN still produces one all-NULL row. Count the joined key instead.

Q2 **Second highest salary, no LIMIT and no window function.**

```
SELECT MAX(salary) AS second_highest
FROM employees
WHERE salary < (SELECT MAX(salary)
                FROM employees);
```

Q3 **Third highest *distinct* salary, ties collapsed.**

```
WITH r AS (
  SELECT salary, DENSE_RANK()
    OVER (ORDER BY salary DESC) rnk
  FROM employees)
SELECT DISTINCT salary
FROM r WHERE rnk = 3;
```

⭐ How To Attack Any Query Question

1. Say the grain out loud ("one row per customer per month") before typing · 2. Pick the driving table — whichever rows must all survive decides INNER vs LEFT · 3. Filter, aggregate, then rank — WHERE → GROUP BY → HAVING → window → ORDER BY · 4. Name your NULLs — missed joins, missing dates and empty groups are where most wrong answers live.

Joins That Trip People Up

Q4 ***List customers who have never placed an order.***

```
SELECT c.cust_id, c.cust_name
FROM customers c
LEFT JOIN orders o ON o.cust_id = c.cust_id
WHERE o.order_id IS NULL;
```

Say this: anti-join. The IS NULL must sit on a column that is never null in a matched row.

Q5 ***Find delivered orders that have no payment row.***

```
SELECT o.order_id, o.amount
FROM orders o
WHERE o.status = 'DELIVERED'
  AND NOT EXISTS (SELECT 1 FROM payments p WHERE p.order_id = o.order_id);
```

Q6 ***MySQL has no FULL OUTER JOIN. Emulate one between customers and orders.***

```
SELECT c.cust_id, o.order_id FROM customers c
  LEFT JOIN orders o ON o.cust_id = c.cust_id
UNION
SELECT c.cust_id, o.order_id FROM customers c
  RIGHT JOIN orders o ON o.cust_id = c.cust_id;
```

UNION (not UNION ALL) removes the duplicated matched rows.

Q7 ***Product IDs that exist in products but were never ordered — set-operator version.***

```
SELECT product_id FROM products
EXCEPT
SELECT product_id FROM order_items;    -- MySQL 8.0.31+
```

Older server? Fall back to NOT EXISTS — EXCEPT and INTERSECT only arrived in 8.0.31.

Q8 ***Build a complete date × category grid so empty combinations still appear.***

```
SELECT d.order_date, p.category, COALESCE(SUM(oi.qty), 0) AS units
FROM (SELECT DISTINCT order_date FROM orders) d
CROSS JOIN (SELECT DISTINCT category FROM products) p
LEFT JOIN orders o ON o.order_date = d.order_date
LEFT JOIN order_items oi ON oi.order_id = o.order_id
LEFT JOIN products pr ON pr.product_id = oi.product_id AND pr.category = p.category
GROUP BY d.order_date, p.category;
```

Q9 ***Bucket every product into a price band using a non-equi join.***

```
SELECT p.product_name, b.band
FROM products p
JOIN price_bands b
  ON p.price >= b.min_price AND p.price < b.max_price;
```

Range joins need half-open bounds (>= low, < high) or boundary prices land in two bands.

> 💡 **Tip**
>
> A filter on the right table belongs in the ON clause of a LEFT JOIN. Move it to WHERE and the join silently turns INNER.

> ⭐ **Interview Tip**
>
> When asked "LEFT JOIN + IS NULL or NOT EXISTS?" — say both work, NOT EXISTS reads better and never duplicates rows if the right side has repeats.

Self, Semi & Anti Joins

Q10 ***Find every employee who earns more than their own manager.***

```sql
SELECT e.emp_name, e.salary, m.emp_name AS manager, m.salary AS mgr_salary
FROM employees e
JOIN employees m ON m.emp_id = e.manager_id
WHERE e.salary > m.salary;
```

Two aliases, one table. The join key flips direction: child.manager_id = parent.emp_id.

Q11 ***List all pairs of colleagues in the same department, without mirror duplicates.***

```sql
SELECT a.emp_name AS emp_a, b.emp_name AS emp_b, a.dept_id
FROM employees a
JOIN employees b ON b.dept_id = a.dept_id AND b.emp_id > a.emp_id;
```

b.emp_id > a.emp_id kills both the self-pair (A,A) and the mirror (B,A) in one condition.

Q12 ***Managers with three or more direct reports.***

```sql
SELECT m.emp_id, m.emp_name, COUNT(*) AS direct_reports
FROM employees m
JOIN employees e ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.emp_name
HAVING COUNT(*) >= 3;
```

Q13 ***Customers who bought product 101 but never product 202.***

```sql
SELECT DISTINCT o.cust_id
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
WHERE oi.product_id = 101
  AND NOT EXISTS (SELECT 1 FROM orders o2
                  JOIN order_items i2 ON i2.order_id = o2.order_id
                  WHERE o2.cust_id = o.cust_id AND i2.product_id = 202);
```

Semi-join for the "did buy", anti-join for the "never bought". Classic two-part ask.

Q14 ***Employees who manage nobody (the leaves of the org tree).***

```sql
SELECT e.emp_id, e.emp_name
FROM employees e
WHERE NOT EXISTS (SELECT 1 FROM employees r WHERE r.manager_id = e.emp_id);
```

Careful with NOT IN: if any manager_id is NULL, `emp_id NOT IN (SELECT manager_id ...)` returns nothing at all.

Q15 ***Show each employee next to their department average, without collapsing rows.***

```sql
-- correlated subquery: re-runs per row
SELECT e.emp_name, e.salary,
       (SELECT AVG(x.salary) FROM employees x WHERE x.dept_id = e.dept_id) AS dept_avg
FROM employees e;
-- window version: one pass, always prefer this
SELECT emp_name, salary, AVG(salary) OVER (PARTITION BY dept_id) AS dept_avg FROM employees;
```

> ⭐ **Interview Tip**
>
> Write the correlated version, then say out loud: "the window version reads the table once instead of once per row." Showing you know the cost of your own query is most of the signal in this round.

Aggregation, GROUP BY & HAVING

Q16 **Departments where the average salary tops 50,000 and headcount is at least 5.**

```sql
SELECT d.dept_name, AVG(e.salary) AS avg_sal, COUNT(*) AS headcount
FROM employees e JOIN departments d ON d.dept_id = e.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING AVG(e.salary) > 50000 AND COUNT(*) >= 5;
```

Q17 **Revenue by product category, highest first.**

```sql
SELECT p.category, SUM(oi.qty * oi.unit_price) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category
ORDER BY revenue DESC;
```

Revenue lives on the line item (qty × unit_price), never on orders.amount, or you double count.

Q18 **Monthly count of distinct buyers, cancelled orders excluded.**

```sql
SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, COUNT(DISTINCT cust_id) AS buyers
FROM orders
WHERE status <> 'CANCELLED'
GROUP BY DATE_FORMAT(order_date, '%Y-%m')
ORDER BY ym;
```

Q19 **Category revenue with a grand-total row at the bottom.**

```sql
SELECT COALESCE(p.category, 'ALL CATEGORIES') AS category,
       SUM(oi.qty * oi.unit_price) AS revenue
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category WITH ROLLUP;
```

WITH ROLLUP adds the subtotal rows and marks them with NULL in the grouped column.

Q20 **Average order value per customer, only for customers with 3+ orders.**

```sql
SELECT cust_id, COUNT(*) AS orders_cnt, ROUND(AVG(amount), 2) AS aov
FROM orders
GROUP BY cust_id
HAVING COUNT(*) >= 3;
```

Q21 **Which weekday brings in the most orders?**

```sql
SELECT DAYNAME(order_date) AS dow, COUNT(*) AS cnt
FROM orders
GROUP BY DAYNAME(order_date)
ORDER BY cnt DESC
LIMIT 1;
```

> 💡 **Tip**
>
> WHERE throws away rows before grouping, HAVING throws away groups after. Anything that can go in WHERE should, because it shrinks the work.

> ⭐ **Interview Tip**
>
> MySQL 8 ships with ONLY_FULL_GROUP_BY on. Every non-aggregated column in SELECT must appear in GROUP BY or be functionally dependent on a grouped primary key.

Conditional Aggregation & Pivots

Q22 *Turn order status into columns: one row per customer, counts per status.*

```
SELECT cust_id,
       SUM(status = 'DELIVERED') AS delivered,
       SUM(status = 'CANCELLED') AS cancelled,
       SUM(status = 'PLACED')    AS placed
FROM orders GROUP BY cust_id;
```

MySQL shortcut: a comparison evaluates to 1 or 0, so SUM() counts it. The portable form is SUM(CASE WHEN status = 'DELIVERED' THEN 1 ELSE 0 END).

Q23 *Revenue by category with 2025 and 2026 side by side.*

```
SELECT p.category,
       SUM(CASE WHEN YEAR(o.order_date) = 2025 THEN oi.qty*oi.unit_price END) AS rev_2025,
       SUM(CASE WHEN YEAR(o.order_date) = 2026 THEN oi.qty*oi.unit_price END) AS rev_2026
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category;
```

No ELSE means unmatched rows contribute NULL, and SUM ignores NULL. That is what you want here.

Q24 *Cancellation rate per customer as a percentage.*

```
SELECT cust_id, ROUND(100 * AVG(status = 'CANCELLED'), 2) AS pct_cancelled
FROM orders GROUP BY cust_id;
```

AVG of a 1/0 flag is the rate. Neat, and it avoids a second COUNT(*) in the denominator.

Q25 *Customers who ordered in both 2025 and 2026.*

```
SELECT cust_id FROM orders
GROUP BY cust_id
HAVING SUM(YEAR(order_date) = 2025) > 0
   AND SUM(YEAR(order_date) = 2026) > 0;
```

Q26 *Unpivot: turn three metric columns of a summary table into label/value rows.*

```
SELECT cust_id, 'delivered' AS metric, delivered AS val FROM cust_summary
UNION ALL SELECT cust_id, 'cancelled', cancelled FROM cust_summary
UNION ALL SELECT cust_id, 'placed',    placed    FROM cust_summary;
```

MySQL has no UNPIVOT operator — UNION ALL is the answer they are listening for.

Q27 *First order, last order and active lifespan per customer.*

```
SELECT cust_id, MIN(order_date) AS first_order, MAX(order_date) AS last_order,
       DATEDIFF(MAX(order_date), MIN(order_date)) AS days_active
FROM orders GROUP BY cust_id;
```

> ⭐ **Interview Tip**
>
> "Pivot this without PIVOT" is one of the most common on-screen tasks for analytics-leaning DE roles. Conditional aggregation is the whole answer: one CASE per output column, wrapped in SUM or COUNT.

Subqueries & Correlated Subqueries

Q28 ***Employees paid above the average of their own department.***

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (SELECT AVG(x.salary) FROM employees x WHERE x.dept_id = e.dept_id);
```

Correlated: the inner query references the outer row (e.dept_id), so it cannot run on its own.

Q29 ***The most recent order for every customer, ties allowed.***

```
SELECT o.*
FROM orders o
WHERE o.order_date = (SELECT MAX(o2.order_date) FROM orders o2
                      WHERE o2.cust_id = o.cust_id);
```

Q30 ***Products priced above their category average.***

```
SELECT p.product_name, p.category, p.price
FROM products p
JOIN (SELECT category, AVG(price) AS avg_price FROM products GROUP BY category) c
  ON c.category = p.category
WHERE p.price > c.avg_price;
```

Derived table computes the averages once, instead of once per product row.

Q31 ***Customers whose lifetime spend beats the average lifetime spend.***

```
SELECT t.cust_id, t.total
FROM (SELECT cust_id, SUM(amount) AS total FROM orders GROUP BY cust_id) t
WHERE t.total > (SELECT AVG(total) FROM
                 (SELECT SUM(amount) AS total FROM orders GROUP BY cust_id) z);
```

Cleaner with a CTE — define the totals once and reference it twice (next page).

Q32 ***Each category and its single best-selling product name.***

```
SELECT p.category,
       (SELECT p2.product_name
          FROM products p2 JOIN order_items oi ON oi.product_id = p2.product_id
         WHERE p2.category = p.category
         GROUP BY p2.product_id, p2.product_name
         ORDER BY SUM(oi.qty) DESC LIMIT 1) AS best_seller
FROM products p GROUP BY p.category;
```

Q33 ***Orders larger than that customer's own average order.***

```
SELECT o.order_id, o.cust_id, o.amount
FROM orders o
WHERE o.amount > (SELECT AVG(o2.amount) FROM orders o2 WHERE o2.cust_id = o.cust_id);
```

> 💡 **Tip**
>
> A scalar subquery must return one row, one column. If it can return two rows, the query dies at runtime, not at parse time.

> ⭐ **Interview Tip**
>
> MySQL will not accept LIMIT inside IN / ANY / ALL subqueries. Wrap it in a derived table: `IN (SELECT id FROM (SELECT ... LIMIT 5) t)`.

EXISTS vs IN vs JOIN

Q34 ***Customers who placed at least one order — write it three ways.***

```
SELECT c.* FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.cust_id = c.cust_id);   -- 1
SELECT c.* FROM customers c WHERE c.cust_id IN (SELECT cust_id FROM orders); -- 2
SELECT DISTINCT c.* FROM customers c JOIN orders o ON o.cust_id = c.cust_id; -- 3
```

Only the JOIN needs DISTINCT, because it multiplies a customer by their order count. That is the whole difference.

Q35 ***Why does this return zero rows, and how do you fix it?***

```
-- broken when any manager_id is NULL
SELECT * FROM employees WHERE emp_id NOT IN (SELECT manager_id FROM employees);
-- fix 1
SELECT * FROM employees WHERE emp_id NOT IN
      (SELECT manager_id FROM employees WHERE manager_id IS NOT NULL);
```

NOT IN with a NULL evaluates to UNKNOWN for every row, so nothing passes. NOT EXISTS is immune.

Q36 ***Departments with at least one employee earning above 100,000.***

```
SELECT d.dept_name FROM departments d
WHERE EXISTS (SELECT 1 FROM employees e
              WHERE e.dept_id = d.dept_id AND e.salary > 100000);
```

EXISTS stops at the first match — it answers yes/no, it does not count.

Q37 ***Orders that contain any Electronics product.***

```
SELECT o.order_id FROM orders o
WHERE EXISTS (SELECT 1 FROM order_items oi
              JOIN products p ON p.product_id = oi.product_id
              WHERE oi.order_id = o.order_id AND p.category = 'Electronics');
```

Q38 ***Customers who have bought every product in the Audio category (relational division).***

```
SELECT o.cust_id
FROM orders o
JOIN order_items oi ON oi.order_id = o.order_id
JOIN products p ON p.product_id = oi.product_id AND p.category = 'Audio'
GROUP BY o.cust_id
HAVING COUNT(DISTINCT p.product_id) = (SELECT COUNT(*) FROM products
                                       WHERE category = 'Audio');
```

"Every / all" questions almost always become COUNT(DISTINCT x) = the size of the full set.

Q39 ***Products bought by more than 10 distinct customers.***

```
SELECT oi.product_id, COUNT(DISTINCT o.cust_id) AS buyers
FROM order_items oi JOIN orders o ON o.order_id = oi.order_id
GROUP BY oi.product_id
HAVING COUNT(DISTINCT o.cust_id) > 10;
```

> ⭐ ***Interview Tip***
>
> If they ask which is faster: in MySQL 8 the optimizer rewrites most IN subqueries into semi-joins, so EXISTS and IN usually plan the same. Answer on correctness first (NULL safety, row multiplication), performance second.

Common Table Expressions

Q40 **Rewrite the "spend above average spend" query with a CTE defined once.**

```
WITH cust_totals AS (
  SELECT cust_id, SUM(amount) AS total FROM orders GROUP BY cust_id)
SELECT cust_id, total FROM cust_totals
WHERE total > (SELECT AVG(total) FROM cust_totals);
```

The whole point: a CTE can be referenced more than once. A derived table cannot.

Q41 **Chain two CTEs: monthly revenue, then only the months above the yearly average.**

```
WITH monthly AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS revenue
  FROM orders GROUP BY 1),
avg_month AS (SELECT AVG(revenue) AS avg_rev FROM monthly)
SELECT m.ym, m.revenue FROM monthly m JOIN avg_month a
WHERE m.revenue > a.avg_rev;
```

Later CTEs can read earlier ones. Read the query top to bottom like a pipeline.

Q42 **Compare every department average against the company average in one pass.**

```
WITH dept_avg AS (
  SELECT dept_id, AVG(salary) AS avg_sal FROM employees GROUP BY dept_id)
SELECT d.dept_id, d.avg_sal,
       (SELECT AVG(avg_sal) FROM dept_avg) AS company_avg,
       ROUND(d.avg_sal - (SELECT AVG(avg_sal) FROM dept_avg), 2) AS gap
FROM dept_avg d ORDER BY gap DESC;
```

Q43 **Filter on a window function result — why WHERE fails and what to do.**

```
-- ERROR: window functions are not allowed in WHERE
SELECT emp_name, RANK() OVER (ORDER BY salary DESC) r FROM employees WHERE r <= 5;
-- correct: compute in a CTE, filter outside it
WITH ranked AS (
  SELECT emp_name, RANK() OVER (ORDER BY salary DESC) r FROM employees)
SELECT * FROM ranked WHERE r <= 5;
```

Windows run after WHERE and GROUP BY, so they need an extra layer before you can filter them.

Q44 **Build a clean order spine, then attach item counts to it.**

```
WITH valid_orders AS (
  SELECT order_id, cust_id, order_date FROM orders WHERE status <> 'CANCELLED')
SELECT v.order_id, v.order_date, COUNT(oi.item_id) AS lines
FROM valid_orders v
LEFT JOIN order_items oi ON oi.order_id = v.order_id
GROUP BY v.order_id, v.order_date;
```

Q45 **Deduplicate first, aggregate second — in one readable statement.**

```
WITH deduped AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id ORDER BY paid_at DESC) rn
  FROM payments)
SELECT SUM(amount) AS collected FROM deduped WHERE rn = 1;
```

> 💡 **Tip**
>
> A CTE is not automatically materialised in MySQL. It can be merged into the outer query, so it is readability first, performance only sometimes.

> ⭐ **Interview Tip**
>
> Name CTEs after the grain they produce — monthly_revenue, first_payment_per_order. Interviewers read your naming as evidence you think in tables, not in steps.

Recursive CTEs

The Shape Never Changes

```
WITH RECURSIVE cte AS (
  -- anchor: where the walk starts
  SELECT ...
  UNION ALL
  -- recursive: joins back to cte
  SELECT ... FROM cte JOIN ...
  WHERE <stop condition>)
SELECT * FROM cte;
```

Three things to say out loud:

- The anchor runs once.
- The recursive arm runs until it returns zero rows.
- MySQL caps depth at cte_max_recursion_depth (1000 by default) so a bad join fails fast instead of hanging.

Q46 *Generate every date in January 2026, with no calendar table.*

```
WITH RECURSIVE d AS (
  SELECT DATE('2026-01-01') AS dt
  UNION ALL
  SELECT DATE_ADD(dt, INTERVAL 1 DAY) FROM d WHERE dt < '2026-01-31')
SELECT dt FROM d;
```

This is the fix for "days with zero orders are missing from my report". LEFT JOIN orders onto this spine.

Q47 *Every employee under manager 5, at any depth, with their level.*

```
WITH RECURSIVE org AS (
  SELECT emp_id, emp_name, manager_id, 1 AS lvl FROM employees WHERE emp_id = 5
  UNION ALL
  SELECT e.emp_id, e.emp_name, e.manager_id, o.lvl + 1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id)
SELECT * FROM org WHERE emp_id <> 5 ORDER BY lvl;
```

Q48 *Print the full reporting path of every employee up to the top.*

```
WITH RECURSIVE path AS (
  SELECT emp_id, emp_name, CAST(emp_name AS CHAR(500)) AS chain
  FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.emp_name, CONCAT(p.chain, ' > ', e.emp_name)
  FROM employees e JOIN path p ON e.manager_id = p.emp_id)
SELECT emp_name, chain FROM path;
```

CAST the anchor column wide enough — MySQL fixes the column type from the anchor row, and a short type truncates every deeper level.

Q49 *How deep does the org chart go?*

```
WITH RECURSIVE org AS (
  SELECT emp_id, manager_id, 1 AS lvl FROM employees WHERE manager_id IS NULL
  UNION ALL
  SELECT e.emp_id, e.manager_id, o.lvl + 1
  FROM employees e JOIN org o ON e.manager_id = o.emp_id)
SELECT MAX(lvl) AS depth FROM org;
```

Q50 *Split 'a,b,c' into one row per value.*

```
WITH RECURSIVE s AS (
  SELECT SUBSTRING_INDEX('a,b,c', ',', 1) AS val,
         SUBSTRING('a,b,c', LENGTH(SUBSTRING_INDEX('a,b,c', ',', 1)) + 2) AS rest
  UNION ALL
  SELECT SUBSTRING_INDEX(rest, ',', 1),
         SUBSTRING(rest, LENGTH(SUBSTRING_INDEX(rest, ',', 1)) + 2)
  FROM s WHERE rest <> '')
SELECT val FROM s;
```

Window Functions — Ranking

Q51 **Show ROW_NUMBER, RANK and DENSE_RANK together and explain the difference.**

```
SELECT emp_name, salary,
       ROW_NUMBER() OVER (ORDER BY salary DESC) AS rn,
       RANK()       OVER (ORDER BY salary DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense
FROM employees;
```

salary	rn	rnk	dense
90,000	1	1	1
80,000	2	2	2
80,000	3	2	2
70,000	4	4	3

ROW_NUMBER — always unique, ties broken arbitrarily unless you add a tiebreaker.
RANK — ties share a rank, then it skips.
DENSE_RANK — ties share a rank, no gaps. Use it for "Nth highest distinct".

Q52 **Rank employees by salary inside each department.**

```
SELECT dept_id, emp_name, salary,
       RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) AS dept_rank
FROM employees;
```

PARTITION BY restarts the numbering per group. It does not filter anything out.

Q53 **The highest-paid employee per department, ties broken by earliest hire date.**

```
WITH r AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY dept_id
                               ORDER BY salary DESC, hire_date ASC) rn
  FROM employees)
SELECT dept_id, emp_name, salary FROM r WHERE rn = 1;
```

A deterministic tiebreaker is the difference between a correct answer and a flaky one. Always add one.

Q54 **Percentile position of each salary within the company.**

```
SELECT emp_name, salary,
       ROUND(PERCENT_RANK() OVER (ORDER BY salary), 3) AS pct_rank
FROM employees;
```

Q55 **Share of employees earning at or below each salary.**

```
SELECT emp_name, salary,
       ROUND(CUME_DIST() OVER (ORDER BY salary), 3) AS cume
FROM employees;
```

PERCENT_RANK starts at 0, CUME_DIST never returns 0 — it includes the current row.

Q56 **Rank categories by revenue — a window on top of an aggregate.**

```
SELECT p.category, SUM(oi.qty * oi.unit_price) AS revenue,
       RANK() OVER (ORDER BY SUM(oi.qty * oi.unit_price) DESC) AS rnk
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category;
```

Legal and common: the window runs after GROUP BY, so it can rank the aggregate itself.

LAG & LEAD — Row to Row

Q57 **Month-over-month revenue change in absolute terms.**

```
WITH monthly AS (
  SELECT DATE_FORMAT(order_date, '%Y-%m') AS ym, SUM(amount) AS revenue
  FROM orders GROUP BY 1)
SELECT ym, revenue,
       revenue - LAG(revenue) OVER (ORDER BY ym) AS mom_change
FROM monthly;
```

The first row is NULL by design — there is no previous month. Say that before they ask.

Q58 **Same thing as a growth percentage, safe against a zero denominator.**

```
SELECT ym, revenue,
       ROUND(100 * (revenue - LAG(revenue) OVER (ORDER BY ym))
             / NULLIF(LAG(revenue) OVER (ORDER BY ym), 0), 2) AS growth_pct
FROM monthly;
```

NULLIF(x, 0) turns a zero denominator into NULL instead of an error or a bogus figure.

Q59 **Days between each customer's consecutive orders.**

```
SELECT cust_id, order_id, order_date,
       DATEDIFF(order_date,
                LAG(order_date) OVER (PARTITION BY cust_id
                                      ORDER BY order_date)) AS days_since_prev
FROM orders;
```

Q60 **Next order date per customer, with a placeholder when there isn't one.**

```
SELECT cust_id, order_date,
       LEAD(order_date, 1, '9999-12-31') OVER (PARTITION BY cust_id
                                               ORDER BY order_date) AS next_order
FROM orders;
```

Third argument is the default for rows that fall off the edge. LAG takes it too.

Q61 **Customers whose most recent order was smaller than the one before it.**

```
WITH seq AS (
  SELECT cust_id, amount, order_date,
         LAG(amount) OVER (PARTITION BY cust_id ORDER BY order_date) AS prev_amt,
         ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date DESC) rn
  FROM orders)
SELECT cust_id, amount, prev_amt FROM seq WHERE rn = 1 AND amount < prev_amt;
```

Q62 **Compare every month against the same month last year.**

```
SELECT ym, revenue,
       LAG(revenue, 12) OVER (ORDER BY ym) AS revenue_ly
FROM monthly;
```

Offset 12 only works if every month exists in the data. Missing months silently shift the comparison — build a date spine first (Q46).

> 💡 **Tip**
>
> LAG and LEAD need an ORDER BY inside OVER(). Without it there is no "previous" row, and MySQL will reject the query rather than guess.

> ⭐ **Interview Tip**
>
> Almost every "month over month", "day over day" or "time between two events" question is LAG plus a subtraction. Aggregate to the right grain first, then lag — lagging raw rows is the usual wrong turn.

Running Totals & Window Frames

Q63 ***Running total of revenue by date.***

```
SELECT order_date, SUM(amount) AS daily,
       SUM(SUM(amount)) OVER (ORDER BY order_date
                              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS running
FROM orders GROUP BY order_date;
```

SUM(SUM(x)) is not a typo — the inner SUM aggregates the day, the outer one accumulates across days.

Q64 ***Running total of spend per customer, oldest order first.***

```
SELECT cust_id, order_date, amount,
       SUM(amount) OVER (PARTITION BY cust_id ORDER BY order_date) AS running_spend
FROM orders;
```

Omit the frame and ORDER BY gives you RANGE UNBOUNDED PRECEDING by default, which is the running total you wanted.

Q65 ***Seven-day moving average of daily revenue, calendar-correct.***

```
SELECT order_date, SUM(amount) AS daily,
       AVG(SUM(amount)) OVER (ORDER BY order_date
            RANGE BETWEEN INTERVAL 6 DAY PRECEDING AND CURRENT ROW) AS ma_7d
FROM orders GROUP BY order_date;
```

ROWS 6 PRECEDING counts rows, so a missing day quietly becomes an 8-day window. RANGE with INTERVAL counts calendar days.

Q66 ***Show the difference between ROWS and RANGE when values tie.***

```
SELECT salary,
       COUNT(*) OVER (ORDER BY salary ROWS UNBOUNDED PRECEDING)  AS by_rows,
       COUNT(*) OVER (ORDER BY salary RANGE UNBOUNDED PRECEDING) AS by_range
FROM employees;
```

RANGE includes every peer with the same ORDER BY value, so tied rows all get the same answer. ROWS stops at the physical row.

Q67 ***Running total that resets at the start of every month.***

```
SELECT order_date, amount,
       SUM(amount) OVER (PARTITION BY DATE_FORMAT(order_date, '%Y-%m')
                         ORDER BY order_date) AS mtd_total
FROM orders;
```

Reset = PARTITION BY. Whatever the total should restart on goes in the partition.

Q68 ***Each order against the biggest order seen so far for that customer.***

```
SELECT cust_id, order_date, amount,
       MAX(amount) OVER (PARTITION BY cust_id ORDER BY order_date
                         ROWS UNBOUNDED PRECEDING) AS max_so_far
FROM orders;
```

Frame Vocabulary

Frame clause	Window it gives you
UNBOUNDED PRECEDING → CURRENT ROW	everything up to now (running total)
n PRECEDING → CURRENT ROW	trailing window of n+1 rows
no ORDER BY at all	the whole partition, every row identical

⭐ Interview Tip

The default frame when you write ORDER BY and no frame clause is RANGE UNBOUNDED PRECEDING, not ROWS. With tied ORDER BY values that quietly changes your running total. Knowing this one default is a senior-level signal.

Buckets, Shares & Medians

Q69 *Split customers into four spend quartiles.*

```
WITH spend AS (
  SELECT cust_id, SUM(amount) AS total FROM orders GROUP BY cust_id)
SELECT cust_id, total,
       NTILE(4) OVER (ORDER BY total DESC) AS quartile
FROM spend;
```

NTILE fills buckets evenly by row count, not by value. Uneven totals still land in equal-sized groups.

Q70 *Each category's share of total revenue, as a percentage.*

```
SELECT p.category, SUM(oi.qty * oi.unit_price) AS revenue,
       ROUND(100 * SUM(oi.qty * oi.unit_price)
             / SUM(SUM(oi.qty * oi.unit_price)) OVER (), 2) AS pct_of_total
FROM order_items oi JOIN products p ON p.product_id = oi.product_id
GROUP BY p.category;
```

OVER () with an empty parenthesis means the whole result set — that is your denominator.

Q71 *Every employee's salary as a share of their department's payroll.*

```
SELECT emp_name, dept_id, salary,
       ROUND(100 * salary / SUM(salary) OVER (PARTITION BY dept_id), 2) AS pct_dept
FROM employees;
```

Q72 *Median salary — MySQL has no MEDIAN function.*

```
WITH r AS (
  SELECT salary,
         ROW_NUMBER() OVER (ORDER BY salary) AS rn,
         COUNT(*) OVER () AS c
  FROM employees)
SELECT AVG(salary) AS median_salary
FROM r WHERE rn IN (FLOOR((c + 1) / 2), CEIL((c + 1) / 2));
```

Averaging the two middle rows handles even counts; for odd counts both expressions point at the same row.

Q73 *The top 20% of customers by spend.*

```
WITH spend AS (
  SELECT cust_id, SUM(amount) AS total FROM orders GROUP BY cust_id),
b AS (SELECT cust_id, total, NTILE(5) OVER (ORDER BY total DESC) AS bucket FROM spend)
SELECT cust_id, total FROM b WHERE bucket = 1;
```

> 💡 **Tip**
>
> Percentages need a denominator you can defend. State whether it is the whole table, the partition or the filtered set before you write the division.

> ⭐ **Interview Tip**
>
> "Top 20%" is ambiguous on purpose. Ask whether they mean the top 20% of customers by count (NTILE) or the customers who make up 80% of revenue (a running total against the grand total). Choosing without asking is the actual mistake.

Top-N Per Group

The single most-asked pattern in a SQL round. Learn one template and bend it.

Q74 Top 3 products by revenue inside each category.

```
WITH rev AS (
  SELECT p.category, p.product_name, SUM(oi.qty * oi.unit_price) AS revenue
  FROM order_items oi JOIN products p ON p.product_id = oi.product_id
  GROUP BY p.category, p.product_id, p.product_name),
r AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY category
                                   ORDER BY revenue DESC) rn FROM rev)
SELECT category, product_name, revenue FROM r WHERE rn <= 3;
```

Aggregate first, rank second. Trying to do both in one SELECT is where people tangle themselves.

Q75 The latest payment row for every order.

```
WITH p AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id
                               ORDER BY paid_at DESC, payment_id DESC) rn
  FROM payments)
SELECT * FROM p WHERE rn = 1;
```

Q76 Top 2 customers by spend in each city.

```
WITH s AS (
  SELECT c.city, c.cust_id, SUM(o.amount) AS total
  FROM customers c JOIN orders o ON o.cust_id = c.cust_id
  GROUP BY c.city, c.cust_id),
r AS (SELECT *, RANK() OVER (PARTITION BY city ORDER BY total DESC) rnk FROM s)
SELECT * FROM r WHERE rnk <= 2;
```

RANK here means ties both survive, so a city can return three rows. Use ROW_NUMBER if they want exactly two.

Q77 Each customer's second most recent order.

```
WITH r AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id
                               ORDER BY order_date DESC) rn
  FROM orders)
SELECT cust_id, order_id, order_date FROM r WHERE rn = 2;
```

Q78 Top 3 salaries per department *without* any window function.

```
SELECT e.dept_id, e.emp_name, e.salary
FROM employees e
WHERE (SELECT COUNT(DISTINCT x.salary) FROM employees x
       WHERE x.dept_id = e.dept_id AND x.salary > e.salary) < 3
ORDER BY e.dept_id, e.salary DESC;
```

Count how many beat you. Fewer than 3 above means you are in the top 3. The pre-8.0 answer, still asked.

Q79 Most recent order per customer, returning the whole row, no CTE.

```
SELECT o.* FROM orders o
LEFT JOIN orders later ON later.cust_id = o.cust_id AND later.order_date > o.order_date
WHERE later.order_id IS NULL;
```

The self anti-join trick: keep the row that has nothing after it.

Gaps, Islands & Streaks

The one idea behind this whole page: for a run of consecutive values, value minus its row number stays constant. Group on that constant and each run collapses into one island. Everything below is that trick wearing different clothes.

Q80 Find the missing order_id ranges in the sequence.

```
SELECT o.order_id + 1 AS gap_start,
       (SELECT MIN(o2.order_id) FROM orders o2 WHERE o2.order_id > o.order_id) - 1 AS gap_end
FROM orders o
WHERE NOT EXISTS (SELECT 1 FROM orders n WHERE n.order_id = o.order_id + 1)
  AND o.order_id < (SELECT MAX(order_id) FROM orders);
```

Q81 Collapse each customer's consecutive ordering days into start/end ranges.

```
WITH d AS (SELECT DISTINCT cust_id, order_date FROM orders),
r AS (SELECT cust_id, order_date,
            ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY order_date) AS rn
     FROM d),
g AS (SELECT cust_id, order_date, DATE_SUB(order_date, INTERVAL rn DAY) AS grp FROM r)
SELECT cust_id, MIN(order_date) AS streak_start, MAX(order_date) AS streak_end,
       COUNT(*) AS days
FROM g GROUP BY cust_id, grp ORDER BY cust_id, streak_start;
```

DISTINCT first. Two orders on the same day would break the row-number arithmetic.

Q82 Longest ordering streak per customer, in days.

```
-- wrap the islands query above as `streaks`
SELECT cust_id, MAX(days) AS longest_streak
FROM streaks GROUP BY cust_id;
```

Q83 Customers with three or more consecutive months of orders.

```
WITH m AS (SELECT DISTINCT cust_id, DATE_FORMAT(order_date, '%Y%m') AS ym FROM orders),
r AS (SELECT cust_id, ym, ROW_NUMBER() OVER (PARTITION BY cust_id ORDER BY ym) rn FROM m),
g AS (SELECT cust_id, ym, PERIOD_DIFF(ym, 200001) - rn AS grp FROM r)
SELECT cust_id, COUNT(*) AS months_in_a_row, MIN(ym) AS from_month
FROM g GROUP BY cust_id, grp HAVING COUNT(*) >= 3;
```

PERIOD_DIFF turns YYYYMM into a month counter, so December to January stops being a gap.

Q84 Collapse a list of consecutive IDs into compact ranges.

```
WITH g AS (
  SELECT id, id - ROW_NUMBER() OVER (ORDER BY id) AS grp FROM seat_bookings)
SELECT MIN(id) AS range_start, MAX(id) AS range_end FROM g GROUP BY grp;
```

> 💡 **Tip**
>
> Dates need DATE_SUB(d, INTERVAL rn DAY) instead of plain subtraction, and months need PERIOD_DIFF. Integers are the only case where value − rn works directly.

> ⭐ **Interview Tip**
>
> If a question mentions consecutive, streak, in a row, continuous or back-to-back, reach for this pattern immediately. Say the name out loud — "this is gaps and islands" — then write it. Naming the pattern is half the credit.

Finding & Killing Duplicates

Q85 *Find every duplicated (order_id, paid_at) combination in payments.*

```
SELECT order_id, paid_at, COUNT(*) AS copies
FROM payments
GROUP BY order_id, paid_at
HAVING COUNT(*) > 1;
```

Q86 *Return one clean row per key, keeping the earliest payment.*

```
WITH r AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY order_id, paid_at
                               ORDER BY payment_id) rn
  FROM payments)
SELECT * FROM r WHERE rn = 1;
```

ROW_NUMBER is the de-dup workhorse: partition by the natural key, order by whichever copy you want to keep.

Q87 *Actually delete the duplicates, keeping the lowest payment_id.*

```
DELETE p FROM payments p
JOIN payments keeper
  ON keeper.order_id = p.order_id
 AND keeper.paid_at  = p.paid_at
 AND keeper.payment_id < p.payment_id;
```

Delete the row that has a smaller twin. No subquery, no temp table, and it survives large tables.

Q88 *Why does this fail, and how do you make MySQL accept it?*

```
-- ERROR 1093: you can't target a table you're also reading from
DELETE FROM payments WHERE payment_id NOT IN
  (SELECT MIN(payment_id) FROM payments GROUP BY order_id, paid_at);
-- fix: hide it behind a derived table
DELETE FROM payments WHERE payment_id NOT IN (
  SELECT keep_id FROM (
    SELECT MIN(payment_id) AS keep_id FROM payments GROUP BY order_id, paid_at) t);
```

Q89 *How many duplicate groups exist, and how many extra rows do they add?*

```
SELECT COUNT(*) AS dup_groups, SUM(copies - 1) AS rows_to_remove
FROM (SELECT order_id, paid_at, COUNT(*) AS copies
      FROM payments GROUP BY order_id, paid_at HAVING COUNT(*) > 1) d;
```

Two different numbers, and interviewers do check that you know which one they asked for.

Q90 *Keep only the newest record per natural key when the table has a version column.*

```
WITH latest AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY cust_id
                               ORDER BY updated_at DESC, version DESC) rn
  FROM customer_snapshots)
SELECT * FROM latest WHERE rn = 1;
```

This is a CDC / SCD-2 pattern in disguise. Expect it in any data engineering loop.

> 💡 **Tip**
>
> Before deleting anything, run the same logic as a SELECT and count the rows. Then wrap the DELETE in a transaction. Nobody has ever regretted checking first.

> ⭐ **Interview Tip**
>
> Ask what makes a row a duplicate before you write anything. Exact copy of every column and same business key, different values are two different problems with two different answers.

NULL Logic That Costs Offers

Q91 *Three counts, three different answers. Explain each.*

```
SELECT COUNT(*)                 AS all_rows,      -- every row
       COUNT(manager_id)        AS non_null_mgr, -- skips NULL
       COUNT(DISTINCT manager_id) AS uniq_mgr      -- skips NULL + dupes
FROM employees;
```

Q92 *Sort salaries descending but push NULLs to the bottom — MySQL has no NULLS LAST.*

```
SELECT emp_name, salary
FROM employees
ORDER BY (salary IS NULL), salary DESC;
```

salary IS NULL returns 0 or 1, so sorting on it ascending sends the NULLs to the end.

Q93 *Show a usable contact value from the first non-null of three columns.*

```
SELECT cust_id, COALESCE(work_phone, mobile, email, 'no contact') AS contact
FROM customer_contacts;
```

COALESCE takes many arguments, IFNULL takes exactly two. Prefer COALESCE — it is standard SQL and portable.

Q94 *Join two tables on a column that contains NULLs and still match them.*

```
SELECT a.id, b.id
FROM stage a JOIN target b ON a.batch_code <=> b.batch_code;
```

<=> is the NULL-safe equal. NULL = NULL is unknown, NULL <=> NULL is true. Every reconciliation query eventually needs it.

Q95 *Why does AVG disagree with SUM / COUNT(*), and how do you make them agree?*

```
SELECT AVG(discount)                    AS avg_ignoring_nulls,
       SUM(discount) / COUNT(*)          AS avg_over_all_rows,
       AVG(COALESCE(discount, 0))        AS avg_treating_null_as_zero
FROM order_items;
```

Aggregates ignore NULL. Decide out loud whether a missing discount means zero or means unknown, then write the version that matches.

Three-Valued Logic — Keep This In Your Head

Expression	Result	Row survives WHERE?
NULL = NULL	UNKNOWN	no
NULL <=> NULL	TRUE	yes
NULL <> 5	UNKNOWN	no
NOT (NULL)	UNKNOWN	no
TRUE OR NULL	TRUE	yes
FALSE AND NULL	FALSE	no
NULL IS NULL	TRUE	yes
CONCAT('a', NULL)	NULL	—

Where Rows Quietly Disappear

- ✗ NOT IN over a nullable column → empty result
- ✗ Right-table filter moved to WHERE → LEFT JOIN becomes INNER
- ✗ col <> 'X' → drops the NULL rows too
- ✗ GROUP BY on a NULL key → one lumped "NULL" bucket
- ✓ NOT EXISTS, or add IS NOT NULL to the inner query
- ✓ Keep right-table filters inside the ON clause
- ✓ col <> 'X' OR col IS NULL
- ✓ COALESCE the key before grouping if NULL means something

⭐ Interview Tip

The three killers, in the order they actually show up: NOT IN with NULLs, a LEFT JOIN filtered in WHERE, and COUNT(column) vs COUNT(*). Rehearse all three out loud.

Date & Time Queries

Q96 *Orders from the last 30 days, written so the index still gets used.*

```
-- slow: wrapping the column in a function blocks the index
SELECT * FROM orders WHERE DATEDIFF(CURDATE(), order_date) <= 30;
-- sargable: keep the column bare on the left
SELECT * FROM orders
WHERE order_date >= DATE_SUB(CURDATE(), INTERVAL 30 DAY);
```

Say the word "sargable" and explain it: the optimizer can only seek an index when the indexed column sits alone on one side.

Q97 *First and last day of each order's month.*

```
SELECT order_id, order_date,
       DATE_FORMAT(order_date, '%Y-%m-01') AS month_start,
       LAST_DAY(order_date)                AS month_end
FROM orders;
```

Q98 *Weekly order counts, weeks starting on Monday.*

```
SELECT DATE_SUB(order_date, INTERVAL WEEKDAY(order_date) DAY) AS week_start,
       COUNT(*) AS orders_cnt
FROM orders
GROUP BY week_start ORDER BY week_start;
```

WEEKDAY() returns 0 for Monday, which makes this the cleanest week-bucketing in MySQL.

Q99 *Customers who placed their first order within 7 days of signing up.*

```
SELECT c.cust_id, c.signup_date, MIN(o.order_date) AS first_order
FROM customers c JOIN orders o ON o.cust_id = c.cust_id
GROUP BY c.cust_id, c.signup_date
HAVING DATEDIFF(MIN(o.order_date), c.signup_date) <= 7;
```

Q100 *Month-one retention: customers who ordered again in the month after their first.*

```
WITH firsts AS (
  SELECT cust_id, DATE_FORMAT(MIN(order_date), '%Y%m') AS cohort
  FROM orders GROUP BY cust_id),
activity AS (
  SELECT DISTINCT cust_id, DATE_FORMAT(order_date, '%Y%m') AS ym FROM orders)
SELECT f.cohort, COUNT(DISTINCT a.cust_id) AS retained_m1
FROM firsts f
JOIN activity a ON a.cust_id = f.cust_id
               AND a.ym = PERIOD_ADD(f.cohort, 1)
GROUP BY f.cohort ORDER BY f.cohort;
```

PERIOD_ADD('202601', 1) gives 202602 and rolls the year correctly, which plain arithmetic on YYYYMM does not.

MySQL Date Functions Worth Memorising

Function	Does what
DATE_ADD / DATE_SUB(d, INTERVAL n UNIT)	shift by day, week, month, year
DATEDIFF(a,b) / TIMESTAMPDIFF(unit,a,b)	days between; or hours, months
LAST_DAY(d) / WEEKDAY(d)	month end; Monday = 0
PERIOD_DIFF / PERIOD_ADD	month arithmetic on YYYYMM

Tip

Use >= start AND < next_start for date ranges. BETWEEN on a DATETIME column silently drops everything after 00:00:00 on the last day.

Interview Tip

Say the time zone question out loud: is order_date stored in UTC or local time? On a data engineering panel that single question often scores higher than the query itself.

Cheat Sheet & What To Do Next

Hear This → Write That

The question says...	Reach for
top N per group	ROW_NUMBER() OVER (PARTITION BY g ORDER BY x DESC), filter rn <= N
Nth highest distinct	DENSE_RANK(), filter = N
running / cumulative	SUM(x) OVER (ORDER BY d)
moving average	AVG(x) OVER (ORDER BY d RANGE INTERVAL n DAY PRECEDING)
vs previous / next	LAG() / LEAD() with a 3rd default argument
consecutive, streak	value − ROW_NUMBER(), then GROUP BY that
never / no matching row	NOT EXISTS (never NOT IN on a nullable column)
all of / every	COUNT(DISTINCT x) = size of the full set
turn rows into columns	SUM(CASE WHEN … THEN … END)
de-duplicate	ROW_NUMBER() over the natural key, keep rn = 1
missing days in a report	recursive date spine, then LEFT JOIN
share of total	x / SUM(x) OVER () or OVER (PARTITION BY g)

Execution Order — Memorise It

FROM / JOIN → WHERE → GROUP BY → HAVING → SELECT + window → DISTINCT → ORDER BY → LIMIT

This single line explains why you cannot use an alias in WHERE, why windows are illegal in WHERE, and why HAVING sees groups while WHERE sees rows.

14-Day Practice Plan

Day 1–3 — Q1–Q33. Re-type every query from memory, no copying.
Day 4–6 — Q34–Q50. Rewrite three of them two different ways each.
Day 7–10 — Q51–Q79. Windows are the highest-yield section in any SQL round. Spend the extra day here.
Day 11–13 — Q80–Q100. Say the pattern name out loud before writing.
Day 14 — Load the schema into a local MySQL, seed 200 fake rows, and run every query you wrote. Reading SQL is not the same as running it.

💡 Three Habits That Separate Strong Candidates

1. State the grain before typing. 2. Say what your query does to duplicates and NULLs, unprompted. 3. Offer the cost: "this scans once", "this re-reads per row". Interviewers are grading the reasoning, not the keystrokes.

⭐ Coming In Volume 2 — Q101 to Q200

Sessionisation and funnels · cohort retention curves · SCD-2 and CDC merges · JSON and regex · EXPLAIN-driven rewrites and indexing · transactions and isolation · data quality checks · plus 25 full scenario problems.

100 questions down. Now go write them without looking.

Save this deck, and come back to it the week before your loop.

👤➕ FOLLOW | 📤 SHARE | 💬 COMMENT | ❤️ LIKE